
Project Onderzoek

Author Milan Posman

Abstract

Dit project richt zich op het ontwikkelen van een communicatiesysteem op basis van LoRa E220-900T22D-modules, waarbij een microcontroller (ESP32,...) als zender en een Raspberry Pi 5 als ontvanger. Naast de configuratie en integratie van de LoRa-modules worden ook een NEO-6M GPS-ontvanger, DS18B20 temperatuursensor en een LDR toegepast.

De LoRa-communicatie is opgezet via UART met een baudrate van 9600 bps, waarbij M0 en M1 IO worden gebruikt om de E220-900T22D-module te kunnen configureren en gebruiken om data te verzenden en ontvangen. Met een 15 cm LoRa-gluestickantenne werd tijdens een veldtest een betrouwbaar maximaal bereik van 1,1 km gerealiseerd. Bij de configuratie wordt onder andere adressen, frequentiekanaal (868 MHz) en de transmissiemodus ingesteld.

Voor het eindproduct zijn alle componenten geïntegreerd in een behuizing. De verzender moet in een regenbestandige behuizing gesoldeerd op een gaatjesprint met batterijvoeding om buiten te kunnen functioneren. De ontvanger moet in een grote behuizing gevoed door netstroom.

Het eindresultaat is een volledig functioneel prototype dat stabiele LoRa-communicatie, GPS-locatiebepaling, lichtintensiteit- en temperatuurmetingen combineert om te tonen in een dashboard en LCD dat met verdere iteraties online beschikbaar kan worden gemaakt,... .

Content

Intro.....	3
1.1 Situering.....	3
1.2 Doelstellingen.....	3
Materiaal en methodes.....	4
1.3 Gebruikte technologie.....	4
1.4 Hardwaretools gebruikt ter ondersteuning.....	4
1.5 Softwaretools gebruikt ter ondersteuning.....	4
Resultaten.....	5
1.6 Verzender: FIXED_TRANSMISSION & E220-900T22D.....	5
1.6.1 GPIO -- Long range signal -- E220-900T22D	5
1.6.2 Software.....	6
1.6.3 Configuratiemogelijkheden.....	6
1.6.4 Algemene configuratiemogelijkheden.....	8
1.6.5 Na configuratie.....	8
1.7 Prototype verzender 1: ESP32.....	9
1.7.1 Componenten	9
1.7.2 Werkwijze: prototype 1 & 2.....	9
1.8 Verzender prototype 2: Wemos D1 mini pro.....	10
1.8.1 Werkwijze.....	10
1.8.2 Conclusie.....	10
1.9 Ontvanger (Raspberry Pi 5)	11
1.9.1 GPIO	11
1.9.2 Enable UART & I2C.....	12
1.9.3 Controle activatie UART via configuratiebestand	12
1.9.4 Autostart Node-RED.....	13
1.9.5 Controle van autostart.....	13
1.9.6 Test Configuratie & Communicatie met E220 (python).....	14
1.10 Eindproduct ontvanger: Raspberry Pi met Node-RED	15
1.10.1 Installatie en gebruik van Node-RED.....	15
1.10.2 Communicatie met LoRa-module via UART	15
1.10.3 Shutdown & Startup.....	17
Conclusie.....	19
Referentielijst.....	20

Intro

1.1 Situering

Dit project is voor het verkennen van goedkope draadloze IoT-communicatie, waarbij long-range datatransmissie centraal staat. LoRa-technologie wordt gebruikt om sensorwaarden over afstanden groter dan 1 km betrouwbaar door te sturen.

De onderzoeksvraag:

"Hoe kan een betrouwbaar long-range IoT-communicatiesysteem worden gerealiseerd met LoRa-technologie, waarbij (sensor)gegevens over afstanden groter dan 1 km worden verzonden?"

1.2 Doelstellingen

Het doel van deze opdracht is het realiseren van een werkend long-range communicatiesysteem met LoRa-technologie. In de beginfase kozen we de benodigde componenten (LoRa-modules, zender, ontvanger, sensoren, antenne, ...).

De opdracht richt zich op:

- Het configureren en testen van de LoRa E220-modules.
- Het integreren van de sensoren (GPS-module (NEO-6M), temperatuursensor (DS18B20) en LDR).
- Het bouwen van een afgewerkt systeem door alle onderdelen in een passende behuizing te plaatsen.
- Het uitvoeren van praktijktests om bereik en betrouwbaarheid te evalueren.

Het uiteindelijke doel is een stabiel en volledig functioneel communicatiesysteem dat langeafstandsdata kan verzenden en ontvangen.

Materiaal en methodes

1.3 Gebruikte technologie

- LoRa (Long Range): draadloze communicatietechnologie voor lage datasnelheden over grote afstanden.
- UART / I2C / OneWire: communicatiestandaarden voor respectievelijk LoRa, LCD en temperatuursensor.

1.4 Hardwaretools gebruikt ter ondersteuning

- Ebyte E220-900T22D: LoRa-transceivermodule gebaseerd op de LLCC68-chip.
- ESP32: microcontroller als verzender, met meerdere UART-poorten en laag energieverbruik.
- Raspberry Pi 5: single-board computer als ontvanger en data-verwerker.
- Neo-6M / NEO-M8N GPS Module: ontvangst van locatiegegevens.
- LDR (Light Dependent Resistor): meting lichtintensiteit
- DS18B20 Temperatuursensor: digitale temperatuursensor voor externe omgevingstemperatuur.
- Gaatjesprint, kabels, soldeerstation en behuizing : voor het bouwen van prototypes.
- Multimeter om correcte verbinding, kortsluitingen, ... te meten

1.5 Softwaretools gebruikt ter ondersteuning

- Visual Studio (PlatformIO) / Arduino IDE: programmeren en uploaden van ESP32-code.
- Node-RED: visualisatie en besturing via dashboard van Raspberry Pi 5.
- Serial Monitor: debuggen van UART-communicatie.
- Ebyte Configuration Tool: controleren en configureren van LoRa-modules.

Resultaten

1.6 Verzender: FIXED_TRANSMISSION & E220-900T22D

Het verkennen van de E220-module en LoRa-technologie biedt een breed inzicht in de mogelijkheden van LoRa-communicatie. Dit hoofdstuk beschrijft de configuratie, werking en toepassing van de EBYTE LoRa E220-module in combinatie met de Wemos D1 Mini Pro als verzender en ESP32 als ontvanger.

De focus ligt op Fixed Transmission binnen het 868 MHz ISM-spectrum. Bij Fixed Transmission wordt elk bericht verzonden met een expliciet bestemmingsadres en kanaal.

1.6.1 GPIO -- Long range signal -- E220-900T22D

LoRa E220	ESP32	Wemos D1 Mini Pro	Opmerkingen
M0	21 / D21	D7	Normale modus (GND) / Configuratie modus (+3.3V)
M1	19 / D19	D6	Normale modus (GND) / Configuratie modus (+3.3V)
TX	RX2 / D16	D2	Communicatie UART
RX	TX2 / D17	D1	Communicatie UART
AUX	AUX: D22/23	D5	Busy status E220
VCC	3.3 V - 5 V	5 V	Voeding
GND	GND	GND	Aarde

Voorbeeldcode met FT_FIXED_TRANSMISSION zie: ````Sender/Wemos_sendFixedTransmission.ino```` & ````Receiver/ESP32_receiveFixedTransmission/ESP32_receiveFixedTransmission.ino````

1.6.2 Software

De software initialiseert de UART-communicatie op 9600 bps en configureert de LoRa-module via AT-achtige commando's die door de library worden afgehandeld.

Softwarebenodigdheden:

- Arduino IDE / Visual studio code (PlatformIO)
- LoRa_E220 Arduino library
- SoftwareSerial (voor Wemos D1 Mini Pro)

1.6.3 Configuratiemogelijkheden

Adressering

Binnen het LoRa E220-systeem beschikt elke module over een uniek logisch adres, bestaande uit twee delen:

- ADDH (Address High byte) +
- ADDL (Address Low byte)

Samen vormen deze bytes het node-adres ADDH + ADDL waarmee modules elkaar kunnen identificeren binnen een netwerk.

Adres van de zender

In deze configuratie werkt de Wemos D1 Mini Pro als zender:

- configuration.ADDH = 0x00;
- configuration.ADDL = 0x03 ;

Het volledige adres van de zender is daarmee 0x0003. Dit adres wordt intern gebruikt door de LoRa-module om de herkomst van verzonden berichten te identificeren.

Het zenderadres wordt niet meegegeven bij fixed transmissie, maar moet wel uniek zijn binnen het netwerk.

Adres van de ontvanger (Destination)

De ontvanger is geconfigureerd met:

- configuration.ADDH = 0x00 ;
- configuration.ADDL = 0x02 ;

Dit resulteert in het adres 0x0002. In de code van de verzender wordt dit aangeduid met:

```
``` #define DESTINATION_ADDL 2 ```
```

Bij Fixed Transmission wordt het bestemmingsadres expliciet opgenomen in elk verzonden bericht. Hierdoor accepteert uitsluitend de LoRa-module met adres 0x0002 het bericht; andere modules op hetzelfde kanaal negeren deze transmissie.

## Relatie tussen zender en ontvanger

Samengevat:

- Zenderadres: 3 (0x0003)
- Bestemmingsadres: 2 (0x0002)

De zender (3) verstuurt data gericht naar de ontvanger (2) op een vast kanaal. Deze peer-to-peer-communicatie voorkomt dat andere nodes binnen hetzelfde frequentiekanaal de data verwerken.

Deze adresseringsmethode is essentieel voor schaalbare LoRa-netwerken waarin meerdere nodes gelijktijdig actief zijn zonder onderlinge interferentie.

Dat betekent dat enkel de ontvanger met een specifiek adres het bericht kan ontvangen en dus de andere apparaten geen tijd verspillen met onnodige berichten te ontvangen.

## Kanaal en frequentie

Men gebruikt in de code:

- Kanaal: `configuration.CHAN = 0x12`
- Frequentieband = basisfrequentie & kanaal `850(.125) MHz + 0x12 MHz = 868(.125) MHz`

Het kanaal bepaalt de exacte frequentie binnen de toegestane ISM-band. Bij Fixed Transmission kan de channel voor een verzender en ontvanger verschillen. De ontvanger kan bijvoorbeeld `configuration.CHAN = 0x18;` worden, terwijl de verzender 0x12 blijft.

Men moet deze code in de verzender als gevolg ook veranderen:

```
``` e220ttl.sendFixedMessage(0, DESTINATION_ADDL, 0x18, msg); ```
```

De functie `e220ttl.sendFixedMessage()` verzendt een bericht naar module met een specifiek adres en kanaal.

1.6.4 Algemene configuratiemogelijkheden

Transmission Mode: ```FT_FIXED_TRANSMISSION```

Ervoor zorgen dat Fixed Transmission geactiveert wordt in de module

RSSI (Received Signal Strength Indicator): ```DISABLED```

RSSI voegt extra informatie toe aan ontvangen berichten over de signaalsterkte. In deze configuratie is RSSI uitgeschakeld omdat:

- De focus ligt op basiscommunicatie en stabiliteit.
- Extra data-overhead wordt vermeden.
- Indien signaalkwaliteit of bereik in detail moet worden geëvalueerd, kan RSSI worden ingeschakeld voor onderzoeksdoeleinden.

LBT (Listen Before Talk): ```DISABLED```

LBT zorgt ervoor dat de module eerst controleert of het kanaal vrij is voordat er wordt uitgezonden. In dit onderzoek is LBT uitgeschakeld omdat:

- Het netwerk bestaat uit een beperkt aantal bekende nodes (4-6 verzenders, 1 ontvanger).
- Kanaalcongestie niet wordt verwacht. Continue en voorspelbare transmissie wordt verwacht.

WOR-periode (Wake-On-Radio) : ```2000 ms```

WOR bepaalt het interval waarin de module uit een laagvermogensmodus ontwaakt om te controleren op inkomende data.

WOR is met name enkel relevant voor batterijgevoede ontvangers en sinds enkel verzenders batterijgevoed worden kan dit standaard blijven.

1.6.5 Na configuratie

Na configuratie permanent opgeslagen wordt in het interne geheugen van de module wordt vervolgens:

- Bij het opstarten een testbericht verzonden.
- Worden elke 5 seconden berichten verzonden met een tijdstempel. Interessant voor te zien of er berichten verloren geraken.
- Kunnen handmatige berichten via de seriële monitor van de Wemos worden ingevoerd en verzonden.

1.7 Prototype verzender 1: ESP32

1.7.1 Componenten

1. Gaatjesprint 50 * 70 mm
2. Microcontroller: ESP32
3. Stijve geleidende draad en jumperkabels
4. LoRa communicatie LoRa-module: E220
5. Multimeter om correcte verbinding en kortsluitingen te meten
6. Soldeerstation en bijhorende hulpmiddelen

1.7.2 Werkwijze: prototype 1 & 2

1. Solderen van female pinheaders

1. Eerst moet worden bepaald waar elk component wordt aangesloten aan de hand van het schema.
2. De soldering start bij de microcontroller, gevolgd door de LoRa-module.
3. Elke verbinding gecontroleerd met een multimeter om correcte verbinding en het ontbreken van kortsluitingen te verifiëren.

2. Bedrading met stijve geleidbare draad, begin met de aansluiting van Vcc & GND

3. Controle verbinding

Na het solderen wordt de verbinding tweemaal gecontroleerd. De controle focust zowel op het vermijden van kortsluitingen als op het garanderen van een betrouwbare elektrische verbinding.

1. ~ 5 minuten na het solderen
2. ~ 24 uur na het solderen, om mogelijke veranderingen van warmte of mechanische spanning te detecteren.

4. Extra isolatie

- Ter bescherming tegen waterschade en het risico op kortsluiting wordt een isolerende coating aangebracht.
- Tijdens dit proces worden gevoelige componenten tijdelijk verwijderd om beschadiging te voorkomen.

1.8 Verzender prototype 2: Wemos D1 mini pro

1.8.1 Werkwijze

De werkwijze voor prototype 2 is identiek aan die van prototype 1, met als voornaamste verschil dat de ESP32 is vervangen door een Wemos D1 mini Pro. Daarnaast wordt gebruikgemaakt van een kleinere gaatjesprint met afmetingen van 40 × 60 mm.

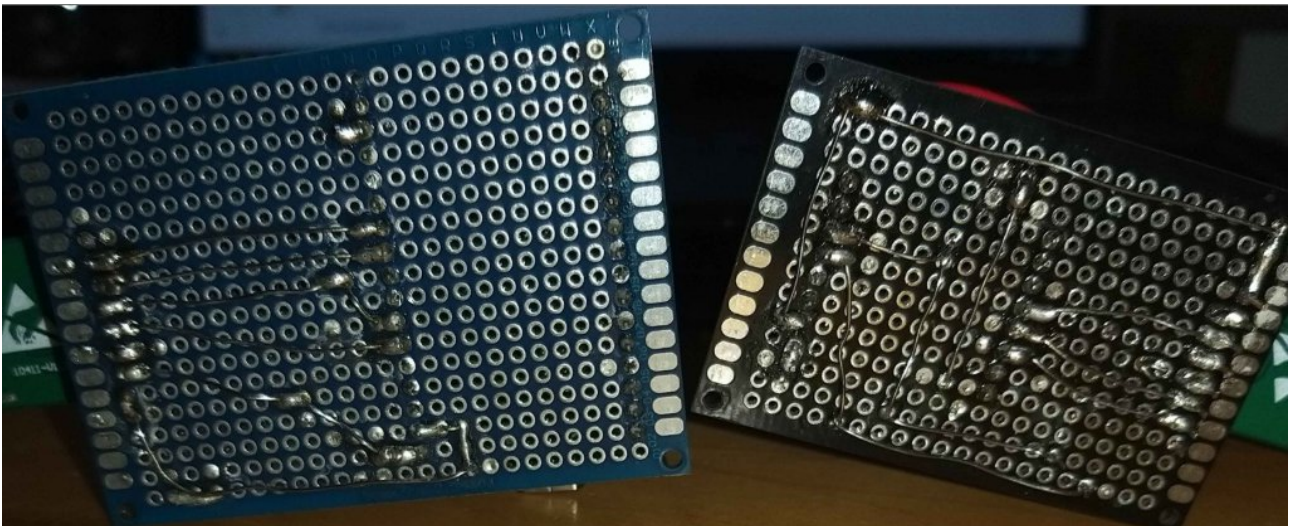
Deze aanpassing heeft als doel te evalueren of het ontwerp kan worden verkleind naar een compacter formaat, zonder negatieve impact op de functionaliteit, betrouwbaarheid of soldeerkwaliteit van het prototype.

1.8.2 Conclusie

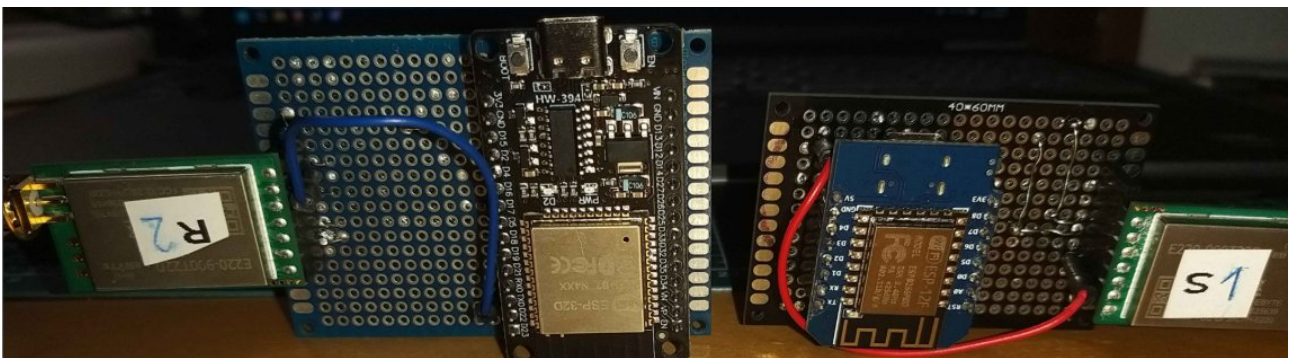
Uit prototype 2 blijkt dat het technisch mogelijk is om het ontwerp verder te verkleinen naar 40 × 60 mm.

Bij dit formaat wordt de integratie van andere componenten, zoals sensoren en batterij veel complexer. De beperkte beschikbare ruimte vereist een zeer doordachte plaatsing en soldering, waarbij manuele assemblage steeds moeilijker wordt.

Om deze reden is dit compact ontwerp beter geschikt voor machinale ontwikkeling, waar hogere precisie kunnen worden gegarandeerd.



Solderen prototype 1 & 2: achterkant



Solderen prototype 1 & 2: voorkant

Om deze 2 prototypes te testen kan het hoofdstuk FT_FIXED_TRANSMISSION geraadpleegd worden.

1.9 Ontvanger (Raspberry Pi 5)

De Raspberry Pi 5 wordt in deze opstelling gebruikt als centrale ontvanger voor draadloze communicatie via de EBYTE LoRa E220-module. Naast het ontvangen van LoRa-berichten vervult de Raspberry Pi ook een verwerkings- en gebruikersinterface geprogrammeerd via Node-RED.

1.9.1 GPIO

De communicatie tussen de Raspberry Pi 5 en de LoRa E220-module verloopt via UART. Volgende tabel geeft een overzicht van de gebruikte GPIO-aansluitingen en hun functie.

LoRa E220 pin	Raspberry Pi 5 aansluiting	Opmerkingen (/dev/ttyAMA0)
VCC	3.3 V – 5 V	Voeding
GND	GND	Aarde
TX	UART RX (GPIO 15)	UART-communicatie
RX	UART TX (GPIO 14)	UART-communicatie
M0	GND / GPIO 21	Normale modus (GND) / Configuratiemodus (+3.3 V)
M1	GND / GPIO 20	Normale modus (GND) / Configuratiemodus (+3.3 V)
AUX	GPIO 18	Busy/statussignaal van de E220-module

Een LCD-scherm wordt aangesloten via het I2C-protocol om ontvangen gegevens visueel weer te geven.

LCD pin	Raspberry Pi 5 aansluiting	Opmerkingen
SCL	SCL (GPIO 3)	I2C communicatie: Serial Clock
SDA	SDA (GPIO 2)	I2C communicatie: Serial Data
GND	GND	Aarde
VCC	5 V	Voeding

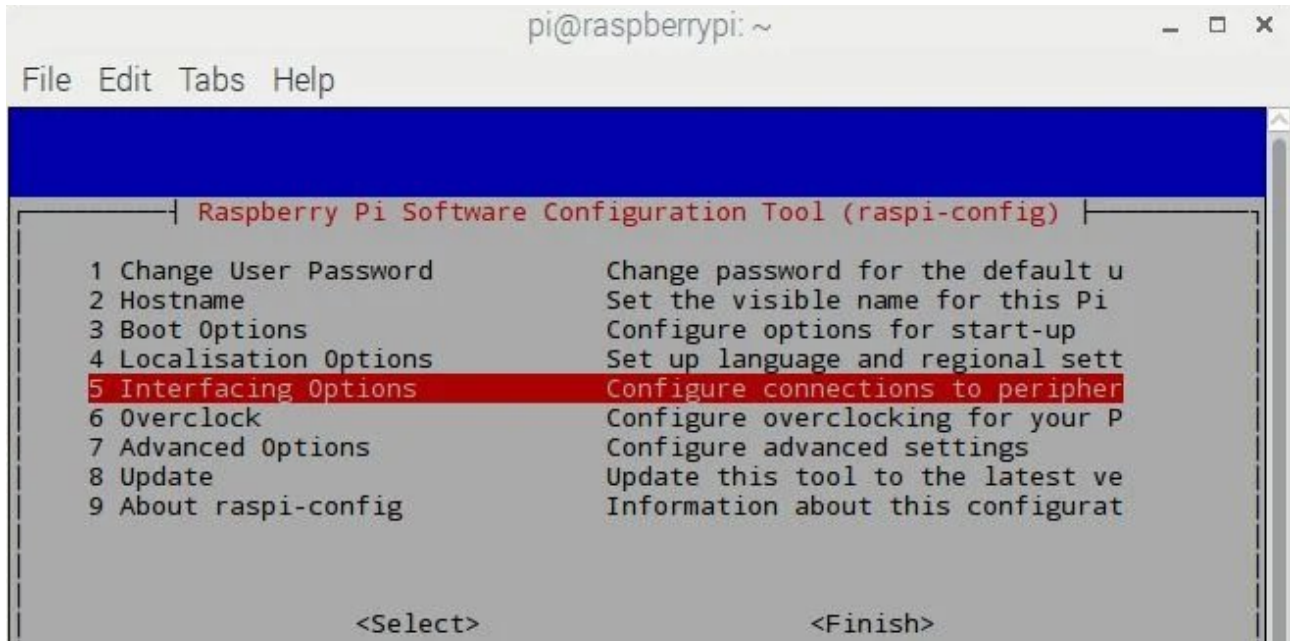
Een drukknop wordt gebruikt om de Raspberry Pi veilig in en uit te schakelen. Daarnaast geeft een LED de operationele status van het systeem en NodeRed weer.

Component	Raspberry Pi 5 aansluiting	Opmerkingen
LED +	GPIO 18	Statusindicatie: Pi actief en Node-RED operationeel (met serieweerstand)
LED –	GND	Aarde LED
Button 0	GPIO 17 & J2	GPIO x voor software-shutdown, J2 voor hardware power control
Button 1	GND (J2 / overige GND)	Aarde knop (normaal open drukknop)

1.9.2 Enable UART & I2C

Om seriële communicatie via UART en I2C mogelijk te maken, dienen deze geactiveerd te worden via de `Interfacing Options` van raspi-config.

Open de Raspberry pi configuratie met het commando `` sudo raspi-config``



Configureer de volgende opties:

- Serial Port
 - Login shell over serial: Disabled
 - Serial port hardware: Enabled
- I2C: Enabled

Bevestig de wijzigingen en herstart de Raspberry Pi.

1.9.3 Controle activatie UART via configuratiebestand

Om te testen dat seriële communicatie via UART geactiveerd is waardoor seriële communicatie via `/dev/tty-AMA0` mogelijk wordt, kan in dit configuratiebestand van de Raspberry Pi gekeken worden.

Aan het einde van `/boot/firmware/config.txt` zal de volgende configuratie aanwezig zijn:

...

[all]

dtoverlay=uart0=on

...

1.9.4 Autostart Node-RED

Om Node-RED automatisch te starten bij het opstarten van de Raspberry Pi en het dashboard fullscreen weer te geven, wordt gebruikgemaakt van een shellscript en een `.desktop`-bestand.

Shellscript (`nodered_autostart.sh`)

...

```
sudo systemctl stop nodered.service;
```

```
sudo systemctl start nodered.service;
```

```
sleep 5;
```

```
chromium --kiosk --password-store=basic "http://127.0.0.1:1880/dashboard"; # open in browser
```

...

De eerste twee commando's herstarten Node-RED. Dit zorgt ervoor dat Node-RED zich steeds in een bekende en stabiele toestand bevindt bij het opstarten van het systeem.

Vervolgens wordt er vijf seconden gewacht om te garanderen dat Node-RED volledig is opgestart voordat het dashboard wordt geopend.

Ten slotte wordt Chromium gestart in kioskmodus, waarbij het Node-RED dashboard automatisch fullscreen op het aangesloten beeldscherm wordt weergegeven. Hierdoor is het systeem geschikt voor visualisatie- of monitoringtoepassingen.

Autostart-configuratie (`~/config/autostart/red.desktop`)

...

[Desktop Entry]

Type=Application

Name=Node-RED Autostart

Exec=<script-location>/nodered_autostart.sh

X-GNOME-Autostart-enabled=true"http://127.0.0.1:1880";

...

Het `.desktop`-bestand in `~/config/autostart/` zorgt ervoor dat het shellscript automatisch wordt uitgevoerd zodra de grafische desktopomgeving van de Raspberry Pi is geladen.

De parameter ``Exec`` verwijst naar de locatie van het shellscript ``nodered_autostart.sh``, terwijl ``X-GNOME-Autostart-enabled=true`` aangeeft dat deze applicatie automatisch wordt opgestart.

1.9.5 Controle van autostart

...

```
chmod +x red.desktop;
```

```
sudo reboot;
```

...

Door het `.desktop`-bestand uitvoerbaar te maken en het systeem te herstarten, kan de correcte werking van de autostart worden gevalideerd.

Na het herstarten start Node-RED automatisch en wordt het dashboard automatisch in kioskmodus weergegeven.

1.9.6 Test Configuratie & Communicatie met E220 (python)

Voor de configuratie en werking van de LoRa-module wordt UART gebruikt in combinatie met M0, M1 en AUX. Door deze signalen correct aan te sturen kan dynamisch worden gewisseld tussen configuratiemodus (AT-commands) en normale transmissiemodus.

Om het wisselen tussen configuratie- en transmissiemodus mogelijk te maken, worden M0 & M1 softwarematig aangestuurd. AUX wordt gebruikt als indicator wanneer de module bezig is.

Voor meer informatie zie ook handleiding E220-xxxTxxx : 5.2 AUX Timing, 7. AT Commands

Pseudocode voor configuratie van receiver:

1. M0 & M1 : 3.3 V / HIGH (Configuration / Sleep mode)
2. Wacht tot AUX uitgang = HIGH (E220 niet bezig)
3. Verzend @-command (UART)
4. Wacht tot AUX = HIGH
5. Lees alles wat E220 beantwoordt (UART)
6. Stap 3 OF exit config M0 & M1 : GND (Transmission mode)
7. Wacht tot AUX = HIGH
8. Ontvang LoRa berichten ...

Het volledig Python-codevoorbeeld voor de ontvanger is terug te vinden in:

`Receiver/reset\_config\_pi5/config+reset.py`

Belangrijke functies in de code:

- `enter\_at\_mode()` / `exit\_at\_mode()`

Wisselen tussen configuratie- en transmissiemodus via GPIO. (at-mode = configuratiemodus)

- `wait\_aux\_ready()`

Blokkering wanneer de module bezig is om timingproblemen te vermijden.

- `send\_at\_command()`

Verzenden en uitlezen van AT-commando's via UART in configuratiemodus.

- `receive\_data()`

Uitlezen van ontvangen in transmissiemodus.

1.10 Eindproduct ontvanger: Raspberry Pi met Node-RED

1.10.1 Installatie en gebruik van Node-RED

Voor de installatie van Node-RED wordt de officiële gids gevolgd: [Node-RED Getting Started – Local](#).

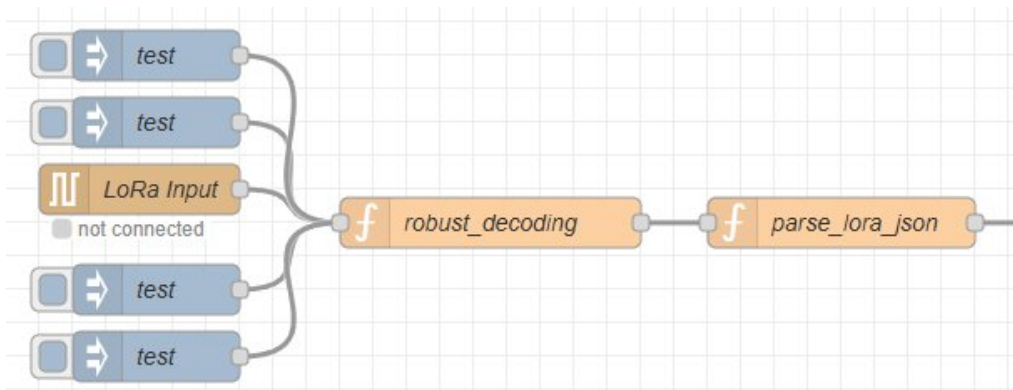
Met een Linux-command (Raspberry Pi,...) kan Node-RED geïnstalleerd worden:

```
``sudo npm install -g --unsafe-perm node-red`` # In Windows wordt sudo niet gebruikt.
```

Na installatie kan Node-RED gestart worden via: ```node-red```

Node-RED kan projecten (flows) exporteren in het .json-formaat, deze kunnen worden geïmporteerd door anderen. Indien het apparaat zich in een netwerk bevindt, kunnen anderen via de website van Node-RED toegang krijgen tot de flows voor controle of aanpassingen.

1.10.2 Communicatie met LoRa-module via UART



UART Communicatie `LoRa Input`

Voor de communicatie tussen Node-RED en de E220-module wordt een `Serial In` node gebruikt. Deze node leest inkomende gegevens via UART van de LoRa-module.

The screenshot shows the 'Edit serial-port node' configuration window. At the top, there are buttons for 'Delete', 'Cancel', and 'Update'. Below is the 'Properties' section. The 'Name' field is set to 'LoRa UART'. The 'Serial Port' field is set to '/dev/ttyAMA0'. The 'Settings' section includes 'Baud Rate' (9600), 'Data Bits' (8), 'Parity' (None), and 'Stop Bits' (1). There are also fields for 'DTR', 'RTS', 'CTS', and 'DSR', all set to 'auto'. The 'Input' section has 'Optionally wait for a start character of' (empty), 'Split input' (on the character), and 'and deliver' (ASCII strings). The 'Output' section has 'Add character to output messages' (empty). The 'Request' section has 'Default response timeout' (10000 ms).

De `seriële poort` wordt `/dev/ttyAMA0` en is de UART-interface via de GPIO pinnen van de Raspberry Pi 5.

Aangezien de Raspberry Pi 5 meerdere UART-poorten heeft, kan een andere seriële poort geselecteerd worden indien er geen data wordt ontvangen. `/boot/firmware/config.txt` kan geraadpleegd worden om de correcte poort te vinden.

`Baud-rate 9600` is vastgelegd door de E220-module.

`Data Bits: 8` is de standaardinstelling voor UART communicatie.

`Geen pariteitscontrole` is de standaardinstelling voor UART communicatie.

`Stop Bits: 1` is de standaardinstelling voor UART communicatie.

UART-data omvormen `Robust decoding`

Voor het omvormen van UART-data wordt een `function` node gebruikt. Deze node zet bytegegevens om naar een leesbaar tekstformaat, zodat verdere verwerking en visualisatie mogelijk worden.

```
let text;
try {
  text = msg.payload.toString('utf8'); // decode bytes
} catch (e) {
  text = JSON.stringify(msg.payload); // fallback
}
msg.payload = text;
return msg;
```

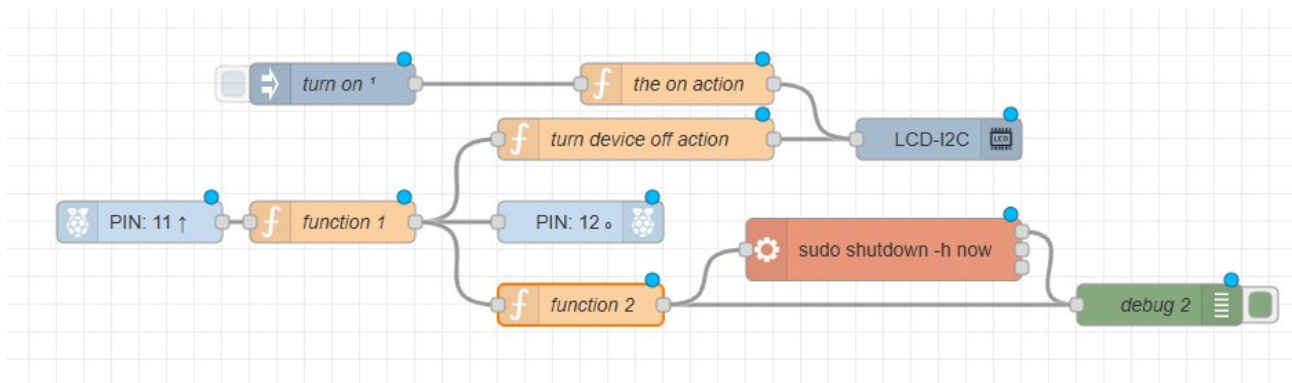
Omdat niet alles naar `UTF-8` omgevormd kan worden (binaire data, transmissiefouten, ...), wordt een fallback-mechanisme gebruikt om de payload alsnog te tonen als `JSON`-data, waardoor debuggen mogelijk blijft.

Dataverwerking `parse\_lora\_json` voor weergave

Deze functie-node verwerkt de eerder gedecodeerde UART-data en zet deze om naar een object dat eenvoudig te gebruiken is in Node-RED.

De identificatie van het apparaat (`device`) en locatiegegevens (`lat, lon, alt, sats`) worden gebruikt in één overzichtelijke object die op een dashboard getoond kan worden. Daarnaast wordt de temperatuur (`tempC`) gecontroleerd. Als `tempC` een geldig getal is, wordt deze afgerond op twee cijfers na de komma en opgeslagen voor weergave.

1.10.3 Shutdown & Startup



GPIO 17 (pin 11):

GPIO 17 wordt gebruikt als invoer voor een actief-lage drukknop. Deze knop initieert het afsluitproces.

Eenmalig afsluiten (`function 1`)

Deze oranje function-node zorgt ervoor dat het afsluitproces slechts één keer kan worden geactiveerd. De functie genereert een signaal dat wordt doorgegeven aan alle relevante flows, zodat randapparatuur correct wordt afgesloten voordat het systeem stopt.

GPIO 18 (pin 12):

GPIO 18 wordt gebruikt als uitgang voor een status-LED. De LED geeft aan dat Node-RED correct is afgesloten en dat de Raspberry Pi is uitgeschakeld. De LED is aangesloten via een serieweerstand zodat de LED niet kapot gaat.

Aansturing LCD via I2C

Om de status van het systeem visueel weer te geven, wordt een LCD-scherm via I2C aangestuurd. Hiervoor wordt de Node-RED LCD-I2C library gebruikt.

De LCD-display kan worden uitgeschakeld (`off action`) en ingeschakeld (`on action`) met een function-node:

```

if(msg.payload == 0){
  var msg = {
    "action": "off", // vervang "off" door "on" om LCD aan te zetten
  };
}
return msg;
  
```

Vertraagde systeemafluiting (`function 2`)

Deze functie initieert de daadwerkelijke uitschakeling van de Raspberry Pi. Hierbij wordt eerst een wachttijd van vijf seconden aangehouden om te garanderen dat alle processen en randapparatuur correct zijn afgesloten.

```
if(flow.get("power") == 0){  
    setTimeout(function() {  
        node.send({ payload: "echo 'Safe test command'" });  
    }, 5000); // 5s delay  
}
```

Voeg `exec` node (rode node) toe met het command: ``sudo shutdown -h now``

Samenvatting van de shutdown-flow (User Experience)

1. Drukknop wordt ingedrukt
2. Node-RED detecteert het signaal van de drukknop via de GPIO-interface.
3. Eenmalige afsluittrigger wordt geactiveerd. Tijdens deze afsluitprocedure wordt:
 - o het LCD-display uitgeschakeld,
 - o de LED uitgeschakeld,
 - o na een wachttijd van 5 seconden het systeemcommando `shutdown -h now` uitgevoerd.

Samenvatting van de startup-flow (User Experience)

1. Drukknop wordt ingedrukt
2. Raspberry Pi start op via de hardwarematige J2-trigger.
3. Het besturingssysteem en de grafische desktopomgeving van de Raspberry Pi worden geladen.
4. Tijdens het opstartproces wordt het shellsript `nodered\_autostart.sh` automatisch uitgevoerd:
 - a. Node-RED service wordt herstart om altijd dezelfde beginstatus te garanderen,
 - b. na een wachttijd van 5 seconden wordt het Node-RED dashboard automatisch geopend in Chromium in kioskmodus.
5. LCD-display wordt gestart met een éénmalig signaal van het blauwe node `turn on`

Conclusie

Dit project heeft aangetoond dat een betrouwbaar long-range IoT-communicatiesysteem gerealiseerd kan worden met de LoRa EBYTE E220-modules in combinatie met microcontrollers (ESP32,...) en een centrale ontvanger (Raspberry Pi 5). In de huidige versie wordt Fixed Transmission nog niet toegepast, maar het testen van dit soort communicatie laat zien dat toekomstige implementaties hiervan mogelijk zijn om peer-to-peer-communicatie met gerichte berichten te realiseren en interferentie tussen meerdere nodes te beperken.

De twee prototypes van de verzender laten zien dat zowel het standaardformaat van de ESP32 (50 × 70 mm) als een compactere uitvoering van de Wemos D1 (40 × 60 mm) technisch realiseerbaar zijn. Het compacte ontwerp beter is voor machinale assemblage en het standaardformaat is beter voor handmatige assemblage.

De Raspberry Pi 5 werkt als centrale ontvanger en verwerkingsstation, waarbij Node-RED de visualisatie en dataverwerking van GPS-, temperatuur- en lichtsensorwaarden mogelijk maakt. De combinatie van UART-communicatie, autostart-scripts en veilige in- en uitschakeling zorgt voor een stabiele en gebruiksvriendelijke werking van het systeem.

Samengevat bewijst dit project dat het mogelijk is om betrouwbaar over afstanden groter dan 1 km te verzenden met een modulair LoRa-systeem, waarbij zowel hardware- als softwarecomponenten effectief samenwerken voor een volledig functioneel long-range communicatiesysteem.

Referentielijst

- [1] PXL DIGITAL, Green Project Research Repository, GitHub. [Online]. Beschikbaar: https://github.com/MilanBowz/PXLDIGITAL_GREEN_PROJECT_RESEARCH.git
- [2] Ebyte, Ebyte LoRa E220-900T22D – Manual + Config Tool. Ebyte, [Online]. Beschikbaar: <https://www.ebyte.com/products/E220-900T22D/4#Downloads/>
- [3] Renzo Mischianti, Ebyte LoRa E220 Device – Specificaties en Basisgebruik. Ebyte, [Online]. Beschikbaar: <https://mischianti.org/ebyte-lora-e220-llcc68-device-for-arduino-esp32-or-esp8266-specs-and-basic-use-1/> [laatste update: 13 December 2023]
- [4] Raspberry Pi Foundation, Enable UART on Raspberry Pi, [Online]. Beschikbaar: <https://www.raspberrypi.org/documentation/configuration/uart.md>
- [5] Arcolinux, <https://arcolinux.com/how-to-autostart-any-application-on-any-linux-desktop/>
- [6] Node-RED, "Getting Started – Local," [Online]. Available: <https://nodered.org/docs/getting-started/local>
- [7] Blackboard PXL, Programming with Node-RED – eBook. Blackboard, [Online]. Beschikbaar: https://blackboard.pxl.be/ultra/courses/_60098_1/outline/file/_2598255_1